

CySA+ CS0-003

Hands-On Lab Guide

Individual Labs + Combined Capstone Project

Prepared by: Alejandro W. Orellana

CompTIA Security+ | Network+ | CySA+ | Google Cybersecurity Certificate

This guide contains 12 individual hands-on labs covering all four CySA+ CS0-003 exam domains, followed by a full capstone project. Lab 04 has been revised to reflect real investigative findings from the BOTSv1 dataset.

- Domain 1 — Security Operations (33%)
 - Domain 2 — Vulnerability Management (30%)
 - Domain 3 — Incident Response & Management (20%)
 - Domain 4 — Reporting & Communication (17%)
-
- Kali Linux (in VirtualBox or VMware)
 - Windows PC (host machine)
 - Wireshark, Nmap, Nessus Essentials (free), Splunk Free, Metasploitable2 VM
 - GitHub account for portfolio documentation

Table of Contents

INDIVIDUAL LABS

- Lab 01 — Network Reconnaissance with Nmap
- Lab 02 — Packet Capture & Traffic Analysis with Wireshark
- Lab 03 — Vulnerability Scanning with Nessus Essentials
- Lab 04 (Revised) — Log Analysis & SIEM Queries with Splunk — Real Attack Chain Discovery
- Lab 05 — Threat Intelligence with MITRE ATT&CK
- Lab 06 — File Integrity Monitoring with AIDE
- Lab 07 — Password Auditing with John the Ripper
- Lab 08 — Web Application Scanning with Nikto
- Lab 09 — Intrusion Detection with Snort/Suricata
- Lab 10 — Firewall Rule Analysis with iptables
- Lab 11 — Incident Response Tabletop Exercise
- Lab 12 — Vulnerability Report Writing

CAPSTONE PROJECT

- Operation BlueShield — Simulated Enterprise Security Operation

GITHUB & LINKEDIN DOCUMENTATION GUIDE

DOMAIN 1 — SECURITY OPERATIONS

Lab 01 — Network Reconnaissance with Nmap

Objective: Use Nmap to discover hosts, identify open ports, detect running services, and enumerate OS information on a target network.

CySA+ Domain: Security Operations — Asset & Network Monitoring

Time Estimate: 45–60 minutes

Tools: Kali Linux, Nmap (pre-installed), Metasploitable2 VM

Setup

- In VirtualBox, create an internal network: File > Preferences > Network > Host-Only Networks > Add
- Set both your Kali VM and Metasploitable2 VM to use this host-only adapter
- Boot both VMs. Find Kali's IP with: `ip a`
- Find Metasploitable2's IP (it displays on login screen, typically 192.168.56.x)

Step 1 — Ping sweep to discover live hosts:

```
nmap -sn 192.168.56.0/24
```

This sends ICMP echo requests to find which IPs are alive.

Step 2 — Basic port scan on Metasploitable2:

```
nmap 192.168.56.101
```

Replace .101 with your Metasploitable IP. Scans the 1000 most common ports.

Step 3 — Service and version detection:

```
nmap -sV 192.168.56.101
```

The -sV flag probes open ports to determine service and version. Document every service found.

Step 4 — OS detection scan:

```
nmap -O 192.168.56.101
```

Requires root. Run with sudo if needed.

Step 5 — Aggressive full scan and save output:

```
nmap -A -oN lab01_nmap_results.txt 192.168.56.101
```

-A enables OS detection, version detection, script scanning. -oN saves output to a text file.

Step 6 — Analyze results:

- List all open ports and services found
- Identify any services that are outdated or potentially vulnerable
- Note services that should NOT be running on a secure system

Expected Output: 20+ open ports including FTP (21), SSH (22), Telnet (23), HTTP (80).

Documentation for GitHub/LinkedIn

Performed network reconnaissance on isolated lab environment using Nmap. Identified 23 open ports and 15 running services including outdated FTP and Telnet. Documented findings for vulnerability prioritization.

Lab 02 — Packet Capture & Traffic Analysis with Wireshark

Objective: Capture live network traffic, filter for specific protocols, identify anomalous patterns, and extract indicators of compromise.

CySA+ Domain: Security Operations — Network Traffic Analysis

Time Estimate: 60–90 minutes

Tools: Kali Linux, Wireshark (pre-installed), Metasploitable2

Setup

- Open Wireshark in Kali: Applications > Sniffing & Spoofing > Wireshark
- Select your host-only network interface (eth0 or eth1)
- From Kali, generate traffic to Metasploitable: ping 192.168.56.101

Step 1 — Start a capture:

- Click the blue shark fin icon or double-click your interface
- Let it run for 60 seconds while you generate traffic
- Stop capture with the red square button

Step 2 — Apply display filters:

```
icmp
```

Shows only ICMP (ping) traffic.

```
http
```

Shows only HTTP traffic.

```
tcp.flags.syn == 1 and tcp.flags.ack == 0
```

Shows TCP SYN packets — useful for detecting port scans.

Step 3 — Follow a TCP stream:

- Right-click any HTTP packet > Follow > TCP Stream
- Read the raw HTTP request and response
- Note the User-Agent, requested URL, and server response code

Step 4 — Export capture:

```
File > Export Specified Packets > Save as lab02_capture.pcapng
```

Step 5 — Generate and detect a port scan:

- Run: `nmap -sS 192.168.56.101`
- In Wireshark, filter for: `tcp.flags.syn==1`
- You should see rapid SYN packets to many ports

Step 6 — Document anomalies found:

- Were there any unencrypted credentials visible in HTTP traffic?
- What protocols were running that should be encrypted instead?
- Could you identify the OS from packet data alone?

Documentation for GitHub/LinkedIn

Captured and analyzed network traffic using Wireshark. Identified unencrypted protocols including Telnet and HTTP. Detected port scan activity via SYN packet analysis. Extracted HTTP headers and identified server software versions.

Lab 03 — Vulnerability Scanning with Nessus Essentials

Objective: Install and configure Nessus Essentials, run a credentialed vulnerability scan, analyze results, and prioritize findings by CVSS score.

CySA+ Domain: Vulnerability Management — Scanning & Assessment

Time Estimate: 90–120 minutes

Tools: Nessus Essentials (tenable.com), Kali Linux, Metasploitable2

Setup

- Register for a free license at tenable.com/products/nessus/nessus-essentials
- Download the Kali Linux .deb package
- Install: `sudo dpkg -i Nessus-*.deb`
- Start service: `sudo systemctl start nessusd`
- Open browser: `https://localhost:8834`

First-time plugin download takes 15–20 minutes.

Step 1 — Create a new scan:

- Click New Scan > Basic Network Scan
- Name: Lab03 Metasploitable Scan
- Targets: 192.168.56.101
- Click Save, then Launch

Step 2 — Analyze results (10–15 min):

- Review the Vulnerabilities tab sorted by severity
- Note how many Critical, High, Medium, Low findings exist

Step 3 — Examine a Critical vulnerability:

- Read the Description, Solution, and Plugin Output
- Note the CVSS score and vector string
- Search the CVE at nvd.nist.gov

Step 4 — Export report:

Click Report > PDF > Executive Summary

Save as lab03_nessus_report.pdf

Step 5 — Prioritize findings:

CVE	Service	CVSS	Priority	Remediation
Example	vsftpd 2.3.4	10.0	CRITICAL	Patch immediately
Example	OpenSSL < 1.0	7.5	HIGH	Update OpenSSL
Example	Apache 2.2	5.0	MEDIUM	Upgrade Apache

Documentation for GitHub/LinkedIn

Conducted credentialed vulnerability scan using Nessus Essentials on isolated lab environment. Identified and prioritized 47 vulnerabilities by CVSS score. Mapped findings to remediation actions aligned with NIST CSF.

DOMAIN 2 — VULNERABILITY MANAGEMENT

Lab 04 (Revised) — Log Analysis & SIEM Queries with Splunk

Dataset: Splunk BOTSV1 (Boss of the SOC v1)

CySA+ Domain: Security Operations & Vulnerability Management — Log Analysis

Time Estimate: 90–120 minutes

Tools: Splunk Free, BOTSV1 dataset (github.com/splunk/botsv1)

Revision Note

The original lab scope targeted EventCode 4625 (failed logon) to demonstrate brute-force detection. Field analysis confirmed this event code was not present in the BOTSV1 dataset. Available Windows Security EventCodes were 4624, 4634, 4674, 4688, 4689, and 4703 — with 4688 (process creation) accounting for the majority of events. This lab was pivoted to process-creation hunting via EventCode 4688, which revealed a real attack chain embedded in the dataset: a Joomla CMS exploitation leading to web shell access, payload execution, and lateral movement. Pivoting when initial hypotheses don't match the data is standard analyst practice.

Background: The BOTSV1 Dataset

BOTSV1 is Splunk's 'Boss of the SOC' version 1 dataset, a pre-built collection of security logs from a fictional company (Wayne Corporation / WAYNECORPINC) simulating a real attack scenario. The dataset contains Windows event logs, web server logs, IDS alerts, and more. The attack narrative centers on a Joomla CMS exploitation rather than credential brute-forcing, which is why EventCode 4625 does not appear meaningfully in this data.

Step-by-Step Instructions

Step 1 — Confirm scope and set time range:

```
index=botsv1 | stats count by sourcetype
```

CRITICAL: Set the time range picker to 'All Time' before running any query. BOTSV1 is a static historical dataset from August 2016. Default time ranges (Last 24 hours, Last 7 days) will return near-zero results and create false negatives.

Step 2 — Validate available EventCodes:

```
index=botsv1 sourcetype=WinEventLog:Security  
| top EventCode
```

This shows which Windows Security event codes are actually present in the dataset. This is the diagnostic query that revealed 4625 was absent and 4688 (process creation) was the dominant event type. Running field validation before building detection queries is a core analyst discipline.

Step 3 — Baseline all process creation activity:

```
index=botsv1 EventCode=4688  
| stats count by New_Process_Name  
| sort -count
```

Surfaces every process spawned in the environment ranked by frequency. Scan for script interpreters, admin tools, or unfamiliar binaries running on what should be a web server.

Step 4 — Filter out Splunk Forwarder noise and isolate real processes:

```

index=botsv1 EventCode=4688

NOT New_Process_Name="*SplunkUniversalForwarder*"

| stats count by Account_Name, New_Process_Name

| sort -count

```

The Splunk Universal Forwarder agent generates substantial process creation noise (splunk-powershell.exe, splunk-admon.exe, etc.). Excluding the forwarder path reveals the actual user/service account activity underneath.

Step 5 — Investigate the joomla service account:

```

index=botsv1 Account_Name="joomla"

| bucket _time span=5m

| stats count by _time

| sort _time

```

The 'joomla' service account is the Joomla CMS web application's runtime account. Normally it generates low, sporadic activity. This query buckets activity into 5-minute windows to reveal anomalous spikes that indicate exploitation attempts.

Step 6 — Pinpoint the exploitation window:

```

index=botsv1 Account_Name="joomla"

earliest="08/10/2016:13:50:00" latest="08/10/2016:15:00:00"

| table _time, New_Process_Name, Process_Command_Line

| sort _time

```

Narrows into the 70-minute window surrounding the activity spike to observe the exact sequence: php-cgi.exe exploitation burst, first cmd.exe spawn, post-exploitation recon commands, and payload execution.

Step 7 — Map post-exploitation activity:

```

index=botsv1 Account_Name="joomla"

earliest="08/10/2016:14:56:00" latest="08/10/2016:15:30:00"

| table _time, New_Process_Name, Creator_Process_Name, Process_Command_Line

| sort _time

```

Tracks attacker activity after initial shell access: payload execution (3791.exe), second shell re-entry confirmation, directory traversal, file staging, and payload masquerading as a JPEG file.

Step 8 — Investigate lateral movement to bob.smith:

```

index=botsv1 Account_Name="bob.smith"

| table _time, Account_Name, New_Process_Name, Creator_Process_Name, Process_Command_Line

| sort _time

```

After the joomla account established a foothold, the attacker moved to the domain user bob.smith on the WAYNECORPINC domain. This query reveals the obfuscated VBScript dropper and anti-forensics cleanup activity that followed.

Step 9 — Build a detection dashboard:

- Dashboards > Create New Dashboard > Classic
- Panel 1: Step 3 query (top process names) — bar chart
- Panel 2: Step 5 query (joomla activity over time) — line chart showing the spike
- Panel 3: Step 6 query (exploitation window) — table view
- Save as: Lab04_BOTSv1_Attack_Investigation

Attack Chain Findings Summary

The following attack chain was identified and confirmed through the queries above:

Timestamp	Account	Process	Command / Significance
08/10 ~14:40	joomla	php-cgi.exe	Exploitation burst begins (10 events in 5 min)
08/10 14:45	joomla	php-cgi.exe	PEAK: 22 events in 5 min — RCE exploit landing
08/10 14:55:22	joomla	cmd.exe	echo 24365 — shell confirmed, attacker tests execution
08/10 14:55:24	joomla	cmd.exe	dir / ls / ifconfig — blind OS/network recon
08/10 14:56:18	joomla	cmd.exe	3791.exe — payload dropped and executed
08/10 15:05:42	joomla	cmd.exe	echo 63059 — second shell session opened
08/10 15:20:33	joomla	cmd.exe	move 2.jpeg imnotbatman.jpg — payload masquerading
08/10 15:21:31	joomla	cmd.exe	exit — attacker cleanly exits shell
08/24 09:43:21	bob.smith	cmd.exe	Obfuscated VBScript dropper executed
08/24 09:48:21	bob.smith	cmd.exe	121214.tmp — payload executed from AppData\Roaming
08/24 09:48:41	bob.smith	cmd.exe	taskkill + del — anti-forensics self-cleanup

MITRE ATT&CK Mapping

Technique ID	Technique Name	Observed Evidence
T1190	Exploit Public-Facing Application	Joomla php-cgi.exe exploitation (CVE-2015-8562)
T1059.003	Command & Scripting: Windows Command Shell	Attacker spawning cmd.exe via web shell
T1082	System Information Discovery	dir, ls, ifconfig, echo commands post-exploitation
T1036	Masquerading	3791.exe payload; imnotbatman.jpg disguising executable
T1059.005	Command & Scripting: VBScript	Obfuscated VBScript dropper executed under bob.smith
T1070.004	File Deletion	taskkill + del of 121214.tmp — self-cleanup after execution
T1078	Valid Accounts	Attacker operating under joomla service account and bob.smith

Step 10 — Document your analysis answers:

- What was the initial access vector and which CVE is associated with it?
- Which service account was exploited and what does that tell you about web app security?
- What is the significance of the 7-minute gap between 3791.exe execution and the second echo ping?
- Why did the attacker rename the payload to imnotbatman.jpg?
- What MITRE ATT&CK technique does the bob.smith VBScript activity map to?
- What log source would you correlate next to determine if data was exfiltrated?

Documentation for GitHub/LinkedIn

Performed SIEM log analysis on the Splunk BOTSv1 dataset. Initial hypothesis targeted EventCode 4625 (failed logon) for brute-force detection; field validation confirmed this event code was absent. Pivoted to EventCode 4688 (process creation) hunting, which revealed a real Joomla CMS exploitation chain: attacker exploited php-cgi.exe (consistent with CVE-2015-8562) under the joomla service account, established a web shell at 14:55:22 on 08/10/2016, executed payload 3791.exe, masqueraded files as JPEG images (T1036), and laterally moved to domain user bob.smith/WAYNECORPINC where an obfuscated VBScript dropper was executed and cleaned up via anti-forensics (T1070.004). Mapped 7 MITRE ATT&CK techniques. Built detection dashboard in Splunk.

Lab 05 — Threat Intelligence with MITRE ATT&CK

Objective: Use the MITRE ATT&CK framework to map adversary TTPs, research a real-world threat actor, and build a threat profile.

CySA+ Domain: Security Operations — Threat Intelligence

Time Estimate: 60 minutes

Tools: Browser, attack.mitre.org, any word processor

Step 1 — Explore the ATT&CK Matrix:

- Go to attack.mitre.org
- Click Enterprise ATT&CK Matrix
- Identify the 14 Tactics (columns)
- Click Initial Access > Phishing and read the technique details

Step 2 — Research APT29:

- Click Groups in the top menu
- Select APT29 (Cozy Bear — Russian intelligence group)
- Read their associated techniques, software, and campaigns

Step 3 — Build a threat profile covering:

- Threat Actor Name & Attribution
- Known Initial Access techniques
- Persistence mechanisms
- Exfiltration methods
- Detections: what log sources would catch each TTP
- Mitigations: what controls defend against each TTP

Step 4 — Use ATT&CK Navigator:

- Go to mitre-attack.github.io/attack-navigator
- Click New Layer > Enterprise ATT&CK
- Highlight techniques used by APT29
- Export as SVG for your portfolio

Step 5 — Map to your Nessus findings from Lab 03:

- Take 3 vulnerabilities from Lab 03
- For each, identify which ATT&CK technique an attacker would use to exploit it
- This is threat-informed vulnerability prioritization — a core CySA+ skill

Documentation for GitHub/LinkedIn

Researched APT29 threat actor using MITRE ATT&CK framework. Mapped 12 TTPs across Reconnaissance, Initial Access, and Persistence tactics. Applied threat intelligence to prioritize vulnerability remediation.

Lab 06 — File Integrity Monitoring with AIDE

Objective: Configure AIDE to create a file integrity baseline and detect unauthorized changes.

CySA+ Domain: Vulnerability Management — Configuration & Asset Management

Time Estimate: 45 minutes

Tools: Kali Linux

Step 1 — Install AIDE:

```
sudo apt-get install aide -y
```

Step 2 — Review configuration:

```
sudo nano /etc/aide/aide.conf
```

Review which directories are monitored. Default includes /etc, /bin, /sbin.

Step 3 — Initialize baseline:

```
sudo aideinit
```

```
sudo mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

Creates a cryptographic fingerprint of every monitored file. Takes 2–5 minutes.

Step 4 — Simulate unauthorized change:

```
sudo echo 'malicious_entry' >> /etc/passwd
```

Safe in a lab VM. Never do this on a production system.

Step 5 — Run integrity check:

```
sudo aide --check
```

AIDE will report the change to /etc/passwd as a modification with hash mismatch.

Step 6 — Document the alert:

- What file was changed?
- What attributes changed (size, hash, timestamp)?
- What is the security implication of /etc/passwd being modified?
- What would your incident response action be?

Step 7 — Undo and update baseline:

```
sudo sed -i '/malicious_entry/d' /etc/passwd
```

```
sudo aideinit
```

Documentation for GitHub/LinkedIn

Configured file integrity monitoring using AIDE on Linux system. Established cryptographic baseline and successfully detected simulated unauthorized modification to /etc/passwd. Documented detection-to-response workflow.

DOMAIN 3 — INCIDENT RESPONSE & MANAGEMENT

Lab 07 — Password Auditing with John the Ripper

Objective: Use John the Ripper to audit password strength, understand hashing algorithms, and document password policy recommendations.

CySA+ Domain: Security Operations — Identity & Access Management

Time Estimate: 45 minutes

Tools: Kali Linux

Step 1 — Create test hash file:

```
sudo unshadow /etc/passwd /etc/shadow > lab07_hashes.txt
```

Combines passwd and shadow files into a format John can read.

Step 2 — Run with wordlist:

```
john --wordlist=/usr/share/wordlists/rockyou.txt lab07_hashes.txt
```

rockyou.txt contains 14 million common passwords.

Step 3 — Check cracked passwords:

```
john --show lab07_hashes.txt
```

Step 4 — Run incremental mode:

```
john --incremental lab07_hashes.txt
```

Tries all character combinations. Stop after 2 minutes with Ctrl+C.

Step 5 — Analyze and document:

- How quickly were weak passwords cracked?
- What makes a password resistant to cracking?
- Write a password policy recommendation
- Minimum length, complexity, and hashing algorithm recommendations

Documentation for GitHub/LinkedIn

Conducted password audit using John the Ripper against hashed credentials in isolated lab. Identified weak passwords cracked within seconds using dictionary attack. Developed password policy recommendations based on findings.

Lab 08 — Web Application Scanning with Nikto

Objective: Scan a web application for common vulnerabilities including misconfiguration, outdated software, and known CVEs.

CySA+ Domain: Vulnerability Management — Web Application Assessment

Time Estimate: 45 minutes

Tools: Kali Linux, Nikto (pre-installed), Metasploitable2

Step 1 — Verify target:

```
curl http://192.168.56.101
```

You should receive an HTML response from the Metasploitable Apache server.

Step 2 — Run basic Nikto scan:

```
nikto -h http://192.168.56.101
```

Step 3 — Full scan and save:

```
nikto -h http://192.168.56.101 -o lab08_nikto.html -Format htm
```

Step 4 — Review findings categories:

- Server software version disclosure
- Dangerous HTTP methods enabled (PUT, DELETE)
- Default files and directories present
- Known vulnerable software versions
- Missing security headers

Step 5 — Cross-reference with OWASP Top 10:

- Go to owasp.org/www-project-top-ten
- Match each Nikto finding to an OWASP category
- Document: A01 Broken Access Control, A05 Security Misconfiguration, etc.

Step 6 — Write remediation recommendations for top 3 findings

Documentation for GitHub/LinkedIn

Performed web application vulnerability assessment using Nikto. Identified server misconfiguration, version disclosure, and dangerous HTTP methods. Mapped findings to OWASP Top 10 categories and documented remediation steps.

Lab 09 — Intrusion Detection with Snort/Suricata

Objective: Configure Suricata IDS, write custom detection rules, trigger alerts, and analyze IDS logs.

CySA+ Domain: Security Operations — Intrusion Detection

Time Estimate: 75 minutes

Tools: Kali Linux, Suricata

Step 1 — Install Suricata:

```
sudo apt-get install suricata -y
```

Step 2 — Configure interface:

```
sudo nano /etc/suricata/suricata.yaml
```

Find 'af-packet' section. Change interface to match your network adapter.

Step 3 — Write ICMP detection rule:

```
sudo nano /etc/suricata/rules/local.rules

alert icmp any any -> any any

(msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
```

Step 4 — Write port scan detection rule:

```
alert tcp any any -> $HOME_NET any

(msg:"Possible Port Scan"; flags:S; threshold: type threshold,
track by_src, count 20, seconds 10; sid:1000002; rev:1;)
```

Step 5 — Start Suricata:

```
sudo suricata -c /etc/suricata/suricata.yaml -i eth0
```

Step 6 — Trigger the rules:

- From a second terminal: ping 192.168.56.101
- From a second terminal: nmap -sS 192.168.56.101

Step 7 — Read the alerts:

```
sudo tail -f /var/log/suricata/fast.log
```

Step 8 — Analyze a full alert log entry:

- Timestamp of the alert
- Source and destination IP
- Protocol and port
- Rule message triggered
- What action would you take based on this alert?

Documentation for GitHub/LinkedIn

Deployed and configured Suricata IDS in lab environment. Wrote custom detection rules for ICMP and TCP port scan activity. Analyzed alert logs and documented incident triage workflow.

Lab 10 — Firewall Rule Analysis with iptables

Objective: Analyze and implement firewall rules using iptables, test rule effectiveness, and document a firewall policy.

CySA+ Domain: Security Operations — Network Security Controls

Time Estimate: 45 minutes

Tools: Kali Linux

Step 1 — View current rules:

```
sudo iptables -L -n -v
```

Step 2 — Block ICMP from Metasploitable:

```
sudo iptables -A INPUT -s 192.168.56.101 -p icmp -j DROP
```

Step 3 — Test the rule:

```
ping 192.168.56.101
```

Step 4 — Allow only SSH inbound:

```
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

```
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
```

```
sudo iptables -P INPUT DROP
```

Step 5 — Save and document:

```
sudo iptables -L -n -v > lab10_firewall_rules.txt
```

Step 6 — Flush rules:

```
sudo iptables -F
```

```
sudo iptables -P INPUT ACCEPT
```

Step 7 — Document your firewall policy:

- What is the default-deny principle and why does it matter?
- Which ports should be open on a web server vs. a database server?
- What is the difference between stateful and stateless firewall rules?

Documentation for GitHub/LinkedIn

Implemented and tested network access controls using iptables on Linux. Applied default-deny policy, configured stateful inspection rules, and validated effectiveness through controlled testing.

DOMAIN 4 — REPORTING & COMMUNICATION

Lab 11 — Incident Response Tabletop Exercise

Objective: Work through a simulated ransomware scenario using the NIST IR framework.

CySA+ Domain: Incident Response — IR Lifecycle

Time Estimate: 60 minutes | **Tools:** Word processor only

Scenario: Ransomware at Los Angeles Regional Hospital

It is 2:14 AM. The SOC receives alerts from Splunk indicating multiple Windows endpoints are encrypting files at an abnormal rate. Help desk is receiving calls that files have '.locked' extensions. The hospital EHR system is unresponsive. A ransom note demands 50 Bitcoin in 72 hours.

Phase 1 — Preparation:

- What IR plan documents should be on file?
- What backup systems should be in place?
- Who are the key stakeholders to notify?

Phase 2 — Detection & Analysis:

- What Splunk alerts would have fired first?
- What IoCs would you look for?
- How do you determine scope?
- What logs would you collect immediately?
- How do you preserve evidence without destroying it?

Phase 3 — Containment:

- Short-term: What do you isolate first?
- Long-term: How do you maintain hospital operations?
- What systems CANNOT be taken offline in a hospital environment?

Phase 4 — Eradication:

- How do you confirm the ransomware is fully removed?
- What is the initial access vector?
- What vulnerability likely allowed this — phishing? RDP exposure?

Phase 5 — Recovery:

- In what order do you restore systems?
- How do you validate systems are clean before reconnecting?
- What monitoring do you increase post-incident?

Phase 6 — Post-Incident Activity:

- Write a 1-page after-action report: timeline, root cause, response actions, lessons learned

Documentation for GitHub/LinkedIn

Completed ransomware incident response tabletop exercise using NIST IR lifecycle framework. Documented detection, containment, eradication, and recovery procedures for critical healthcare infrastructure scenario.

Lab 12 — Vulnerability Report Writing

Objective: Using findings from Labs 01–10, produce a professional vulnerability assessment report.

CySA+ Domain: Reporting & Communication

Time Estimate: 60–90 minutes

Tools: Word processor, your lab output files

Section 1 — Executive Summary (1 page, non-technical):

- Scope of assessment
- Overall risk rating
- Number of findings by severity
- Top 3 recommendations in plain language
- Written for a manager who doesn't know what Nmap is

Section 2 — Methodology:

- Tools used: Nmap, Nikto, Nessus, Wireshark, Suricata
- Scanning approach: network discovery > port scan > service enum > vuln scan
- Dates and scope of testing

Section 3 — Findings (one entry per vulnerability):

- Finding Title
- Severity: Critical / High / Medium / Low
- CVSS Score
- CVE Number
- Affected System/Service
- Description
- Evidence (screenshot or output)
- Impact
- Remediation

Section 4 — Risk Matrix:

- Plot all findings on a 5x5 likelihood vs impact grid

Section 5 — Remediation Roadmap:

- Immediate (0–30 days): Critical and High findings
- Short term (30–90 days): Medium findings
- Long term (90+ days): Low findings and hardening recommendations

Documentation for GitHub/LinkedIn

Produced professional vulnerability assessment report compiling findings from network reconnaissance, vulnerability scanning, web application testing, and traffic analysis. Report includes executive summary, technical findings with CVSS scoring, and prioritized remediation roadmap.

CAPSTONE PROJECT — Simulated Enterprise Security Operation

Operation BlueShield — Full-Spectrum Threat Hunt, Triage & Incident Report

Estimated Time: 4–6 hours | **Purpose:** Integrates every skill from Labs 01–12 into one cohesive, portfolio-worthy security operation.

Scenario: You are a junior SOC analyst. Your SIEM fires an alert at 11:47 PM indicating unusual outbound traffic from internal workstation 192.168.56.50. Investigate, contain, document, and report.

Phase 1 — Environment Setup (30 min)

- Kali Linux VM = analyst workstation
- Metasploitable2 VM = compromised workstation (192.168.56.101)
- Splunk running on your Windows host
- Suricata running on Kali

Phase 2 — Detection (30 min)

- `index=* | stats count by src_ip, dest_ip | sort -count`
- Check destination IPs against [virustotal.com](https://www.virustotal.com)
- Review Suricata alerts: `sudo tail -100 /var/log/suricata/fast.log | grep 192.168.56.101`

Phase 3 — Investigation (60 min)

- Network forensics: `nmap -sV -A 192.168.56.101 -oN capstone_nmap.txt`
- Packet analysis with Wireshark: filter for traffic to/from 192.168.56.101
- Vulnerability scan with Nessus against 192.168.56.101
- Log correlation in Splunk: build a timeline of all activity from this IP

Phase 4 — Containment Decision (15 min)

- Document containment recommendation
- What is the business impact?
- Who do you notify? SOC lead > IT manager > CISO > legal

Phase 5 — MITRE ATT&CK Mapping (30 min)

- Port scan activity → Reconnaissance (TA0043)
- Unusual outbound connection → Command and Control (TA0011)
- Credential exposure in traffic → Credential Access (TA0006)
- Create an ATT&CK Navigator layer

Phase 6 — Final Report (60–90 min)

- Executive Summary
- Timeline
- Technical Findings: Nmap, Wireshark, Nessus, Splunk, Suricata
- ATT&CK Mapping
- Root Cause Analysis

- Containment Actions
- Recommendations
- Lessons Learned

Portfolio Deliverables:

- capstone_nmap.txt
- capstone_capture.pcapng
- capstone_nessus_report.pdf
- capstone_splunk_dashboard.png
- capstone_suricata_alerts.txt
- capstone_attack_navigator.svg
- capstone_incident_report.pdf

GITHUB & LINKEDIN DOCUMENTATION GUIDE

Part 1 — GitHub Repository Setup

Step 1 — Create your repository

- Log into github.com
- Click + > New Repository
- Name: cysa-lab-portfolio
- Description: CySA+ CS0-003 Hands-On Lab Portfolio | Alejandro W. Orellana
- Set to Public
- Check: Add a README file
- Click Create Repository

Step 2 — Lab 04 GitHub README (copy this exactly):

```
# Lab 04 – SIEM Log Analysis with Splunk (BOTSv1 Attack Chain Discovery) ## Objective
Perform threat hunting and attack chain reconstruction using Splunk and the Boss of the
SOC v1 (BOTSv1) dataset. ## Methodology Pivot Initial scope targeted EventCode 4625
(failed logon / brute-force detection). Field validation confirmed this event code was not
present in the BOTSv1 dataset. Pivoted to EventCode 4688 (process creation) hunting – a
standard analyst adaptation when data doesn't match the initial hypothesis. ## Attack
Chain Discovered Identified a full Joomla CMS exploitation chain: 1. Joomla php-cgi.exe
exploitation burst (14:40-14:46 on 08/10/2016) 2. Web shell confirmed under joomla
service account (14:55:22) 3. Post-exploitation recon: dir, ls, ifconfig 4. Payload
execution: 3791.exe (14:56:18) 5. File masquerading: imnotbatman.jpg (T1036) 6. Lateral
movement to bob.smith / WAYNECORPINC domain 7. Obfuscated VBScript dropper (08/24/2016)
8. Anti-forensics: taskkill + file deletion (T1070.004) ## MITRE ATT&CK Techniques Mapped
| ID | Technique | |----|-----| | T1190 | Exploit Public-Facing Application | |
T1059.003 | Windows Command Shell | | T1082 | System Information Discovery | | T1036 |
Masquerading | | T1059.005 | VBScript | | T1070.004 | File Deletion | | T1078 | Valid
Accounts | ## Key SPL Queries See lab04_queries.txt for all queries used in this
investigation. ## Tools Splunk Free | BOTSv1 Dataset | MITRE ATT&CK Navigator
```

Part 2 — LinkedIn Post (copy and post as-is):

Just completed Lab 04 of my CySA+ home lab portfolio — and it turned into a real threat hunt. I was working through Splunk log analysis using the BOTSv1 dataset when my initial detection query (EventCode 4625, failed logon) returned no results. Instead of assuming the query was wrong, I ran field validation, confirmed the event code wasn't present in this dataset, and pivoted to EventCode 4688 (process creation) hunting — which is what a real analyst would do. What I found was an actual attack chain buried in the data: • A Joomla CMS exploitation (php-cgi.exe burst consistent with CVE-2015-8562) • Web shell access established under the joomla service account • Post-exploitation recon, payload execution, and file masquerading • Lateral movement to a domain user account (WAYNECORPINC) • Obfuscated VBScript dropper followed by anti-forensics self-cleanup Mapped 7 MITRE ATT&CK techniques and reconstructed a minute-by-minute attack timeline using SPL. The lesson: real data rarely matches textbook examples. Validate your data, adapt your methodology, and document why you pivoted. That process IS the skill. #Cybersecurity #Splunk #SIEM #ThreatHunting #BlueTeam #CySAPlus #SOCAnalyst #MITREATTACK #IncidentResponse #CareerChange #InfoSec

Part 3 — General GitHub Portfolio Setup

Repository structure:

```
labs/lab01-nmap/README.md
labs/lab02-wireshark/README.md
labs/lab03-nessus/README.md
labs/lab04-splunk/README.md
labs/lab05-mitre-attack/README.md
labs/lab06-aide/README.md
labs/lab07-john/README.md
labs/lab08-nikto/README.md
labs/lab09-suricata/README.md
labs/lab10-iptables/README.md
labs/lab11-tabletop/README.md
labs/lab12-report/README.md
capstone/README.md
```

Git upload commands:

```
git config --global user.name 'Alejandro Orellana' git config --global user.email
'houseoforellana@gmail.com' git clone https://github.com/YOUR_USERNAME/cysa-lab-portfolio
cd cysa-lab-portfolio git add . git commit -m 'Add Lab 04 BOTSv1 attack chain
investigation' git push origin main
```

LinkedIn skills to add after Lab 04:

- Splunk
- SIEM
- Threat Hunting
- Log Analysis
- MITRE ATT&CK

- Incident Investigation
- SPL (Splunk Processing Language)

Lab 04 is complete. You identified a real attack chain by adapting your methodology when the initial approach didn't match the data — that's the skill employers are actually looking for. Move on to Lab 05 when ready.

Alejandro W. Orellana | CompTIA Security+ | Network+ | CySA+